

Detailed Test Design and Execution Report

Software Testing Final Report

Course: Software Testing

Task Management Platform

Team Members: Huang Yihe (2351269)
Guan Jingwen (2352744)
Huang Yihan (2351271)
Xia Haobo (2352048)

June 2026

Contents

1	Project Information	3
2	Test Object	3
3	Test Design Approach	3
3.1	Black-Box Test Design	3
3.1.1	Equivalence Partitioning	4
3.1.2	Boundary Value Analysis	4
3.1.3	Decision Table	4
3.2	White-Box Test Design	4
3.2.1	Path Coverage	5
3.2.2	State Transition	5
4	Test Script Organization	5
5	Tool Implementation and Execution Method	6
6	Execution Results	6
6.1	Original Automatic Script	6
6.2	Additional Explanation of Abnormal Observations	6
6.3	Reviewed Automatic Script	7
6.4	Manually Written Script	8
7	Explanation of Abnormal Observations	8
7.1	Why REQ-005 Path Coverage Changed from 43.8% to 100%	8
7.2	Why REQ-007 Does Not Generate EP/BVA Test Cases	10
8	Comparison Between Final Automatic and Manual Scripts	12
9	Mapping to Known Defects	12

10 Performance Comparison of Different Models	13
11 Generalizability of the System	14
12 Conclusion	15

1 Project Information

Project name: Task Management Platform

Selected module: Task Management module (REQ-005, REQ-006, REQ-007)

2 Test Object

This detailed testing round selects the task management module as the main object. It covers the following three requirements.

Requirement	Content	Risk Level	Test Priority
REQ-005	Task management	Medium	Medium
REQ-006	Task assignment	Medium	Medium
REQ-007	Task statistics	Low	Low

This module was selected for two reasons. First, it includes several types of behavior, such as creation, update, deletion, assignment, and statistics, which makes it suitable for demonstrating both black-box and white-box testing. Second, the module contains several known requirement deviations, which helps observe the effectiveness and limitations of the automated test design tool in a realistic setting.

3 Test Design Approach

In this round, AutoTestDesign first reads the standard requirements, the requirement-code mapping, and the risk analysis results. It then generates black-box and white-box testing assets for REQ-005 to REQ-007. After that, the original automatically exported script is retained, and a manually reviewed automatic script and a manually written comparison script are prepared. These three scripts are used to compare the differences among the original tool output, the reviewed tool output, and manual testing.

3.1 Black-Box Test Design

The black-box part uses three techniques: equivalence partitioning, boundary value analysis, and decision table testing.

3.1.1 Equivalence Partitioning

Equivalence partitioning is mainly applied to the task title, priority, status, and assignee fields.

Field	Valid Classes	Invalid Classes
title	Non-empty and within the allowed length	Empty or too long
priority	Low / Medium / High	Illegal enumeration value
status	todo / in_progress / done	Illegal enumeration value
assignee	Valid assignee input	Empty value or unregistered user

3.1.2 Boundary Value Analysis

Boundary value analysis mainly focuses on `title` length and date input.

Check Point	Representative Values
Around minimum length	1, 2
Around maximum length	199, 200, 201
Date boundary	Past date

3.1.3 Decision Table

Decision table testing is mainly used for condition combinations in task creation, task assignment, and task statistics.

Requirement	Typical Conditions
REQ-005	Whether the project exists, whether the priority is valid
REQ-006	Whether assignee input exists, whether assignment conditions are satisfied
REQ-007	Whether the project exists, whether the statistics request is valid

3.2 White-Box Test Design

The white-box part uses two techniques: path coverage and state transition testing.

3.2.1 Path Coverage

Path coverage mainly targets the following code paths.

Path	Focus
Task creation path	Project not found, invalid title, successful creation
Task update path	Task not found, successful update, invalid input
Task deletion path	Successful deletion, deletion of completed task
Task detail path	Task exists / task does not exist

3.2.2 State Transition

State transition testing models the task lifecycle, including task creation, task viewing, task update, task deletion, and statistics query. This design covers process behavior beyond simple field validation.

4 Test Script Organization

This round keeps three test scripts.

File	Role
test_scripts/test_generated.py	Original automatically exported script
test_scripts/test_generated_refined.py	Automatically generated script after manual review
test_scripts/test_human.py	Manually written comparison script

The latter two scripts keep the same basic structure as the original automatic script. They are still organized by equivalence partitioning, boundary value analysis, decision table, path coverage, and state transition, instead of only keeping scattered examples.

In terms of scale, the number of test functions in each script is:

Script	Number of Test Functions
Original automatic script	44
Reviewed automatic script	32
Manually written script	27

5 Tool Implementation and Execution Method

All tests are executed with Pytest and Flask `test_client()`.

The execution commands are:

```
python -m pytest test_scripts/test_generated.py -v --tb=short
python -m pytest test_scripts/test_generated_refined.py -v --tb=short
python -m pytest test_scripts/test_human.py -v --tb=short
```

This method has three advantages:

1. It does not require starting an additional HTTP service.
2. It can directly call Flask routes inside the process.
3. It is convenient for comparing execution results across script versions.

6 Execution Results

6.1 Original Automatic Script

The original automatically exported script `test_generated.py` contains 44 test functions. The execution result is:

Metric	Value
Total cases	44
Passed	27
Failed	17
Pass rate	61.36%

From the execution log, the original automatic script contains many script-level problems, such as incorrect handling of `null`, wrong endpoint mapping, wrong HTTP methods, and missing preconditions. This shows that the original script generated by the tool provides a useful testing direction, but it cannot be used directly as the final experiment script.

6.2 Additional Explanation of Abnormal Observations

Two observations in the task management module are not immediately intuitive and need to be explained with the execution artifacts and the actual code:

1. The path coverage rate of REQ-005 is only **43.8%**.
2. In the black-box testing of REQ-007, **EP/BVA did not generate test cases**.

These two points are not random. They are caused jointly by the requirement expression, code mapping, and the current implementation strategy of the tool.

6.3 Reviewed Automatic Script

On the basis of preserving the original test design structure, the automatic script was manually reviewed and corrected, producing `test_generated_refined.py`.

The manual corrections mainly focused on four aspects:

1. Correcting wrong endpoints.
2. Correcting wrong HTTP methods.
3. Unifying request data formats.
4. Adding necessary precondition data creation steps.

After correction, the execution result is:

Metric	Value
Total cases	32
Passed	29
Failed	3
Pass rate	90.63%

The three failed cases are:

Failed Case	Type	Corresponding Issue
<code>test_ep_req_006_unregistered_assignee</code>	Real defect	BUG-7
<code>test_bva_req_005_due_date_in_past</code>	Real defect	BUG-9
<code>test_pc_req_005_delete_done_task</code>	Real defect	BUG-8

The reviewed automatic script no longer contains failures caused by the test script itself. All failures directly correspond to real requirement deviations in the task management module of the target application.

6.4 Manually Written Script

For comparison, a manual version `test_human.py` was also written. It keeps the basic black-box and white-box structure, but its coverage scope is more conservative.

The execution result is:

Metric	Value
Total cases	27
Passed	26
Failed	1
Pass rate	96.30%

The failed case is:

Failed Case	Type	Corresponding Issue
<code>test_bva_req_005_due_date_in_past</code>	Real defect	BUG-9

From the result, the manual script only stably discovers BUG-9 and does not cover BUG-7 or BUG-8.

7 Explanation of Abnormal Observations

7.1 Why REQ-005 Path Coverage Changed from 43.8% to 100%

In the first generation result for REQ-005, the tool reported 73 related lines, 32 covered lines, and a path coverage rate of only **43.8%**. The result also showed two uncovered code segments: the project task list API `project_tasks_list()` and the single task detail API `task_detail()`.

At first glance, this looked like the white-box generation failed to cover the complete code paths. However, after comparing the requirement description with the code, I think the more important reason was that the original REQ-005 description was not clear enough. It emphasized task creation, update, deletion, and field validation such as `title`, `priority`, and `status`, but it did not clearly describe read-only paths such as task list query and task detail query. As a result, when the tool generated path coverage cases, it naturally focused on the write-operation paths that were more explicitly stated in the requirement, and did not treat the list and detail APIs as equally important coverage targets.

覆盖率 ↔

关联行

73

覆盖行

32

覆盖率

43.8%

2 代码段未覆盖

routes/tasks.py L25-37 列出项目下的所有任务

routes/tasks.py L61-68 获取单个任务详情

Figure 1: Original REQ-005 path coverage result

覆盖率 ↔

关联行

73

覆盖行

73

覆盖率

100%

Figure 2: REQ-005 path coverage after updating the requirement description

This also shows that a white-box coverage result should not be judged only by the final percentage. It should be traced back to the requirement-code mapping. REQ-005 was mapped to a relatively large range of task route code, and coverage was counted by code lines. If some behavior is mapped into the requirement but is not clearly described in the requirement text, the generated cases may still fail to cover it. Therefore, the 43.8% result reveals a mismatch between the requirement description and the coverage target, rather than simply proving that the code cannot be covered.

To verify this judgment, I revised the REQ-005 requirement description by explicitly adding task list query and task detail query to the behavior scope. After regenerating the path coverage result, the number of related lines remained 73, while the covered lines increased to 73 and the coverage rate became **100%**. This indicates that the earlier low coverage was not caused by the tool being unable to cover these paths, but by an incomplete requirement description that led to incomplete generation targets.

This process gives an important lesson for the experiment: the quality of automated test design

depends strongly on the quality of requirement input. For a requirement like REQ-005 that contains multiple API behaviors, it is not enough to only write "task management" or only emphasize write operations. The boundary among creation, update, deletion, list query, and detail query should be made clear. Otherwise, the generated coverage may look low, but the real issue may be the requirement description and test target rather than the code or the tool itself.

7.2 Why REQ-007 Does Not Generate EP/BVA Test Cases

The `run_results/REQ005_006_007` directory contains the following artifacts for REQ-007:

- REQ-007 判定表.csv
- REQ-007 状态转换 1.csv
- REQ-007 状态转换 2.csv
- REQ-007 路径覆盖.png

However, it does not contain equivalence partitioning or boundary value analysis tables for REQ-007. This is not an export omission. The EP/BVA cases were not effectively generated during the generation stage.

The first reason comes from the requirement itself. In `data/sample_input/task_platform_reqs.csv`, REQ-007 is defined very simply:

- `input_fields` is an empty array.
- `conditions` is an empty array.
- `expected_behaviors` only contains one item: "exists -> return statistics" .

The actual screenshot REQ-007 标准需求生成.png also shows:

- The input field area is empty.
- The related code is concentrated in `project_stats()`.
- The conditions and behavior are basically only "exists -> return" .

This matches the actual code structure of REQ-007. `project_stats()` is essentially a read-only statistics API in `target_app/flask_app/routes/projects.py`.

Its main logic has only two layers:

1. Whether the project corresponding to `project_id` exists.
2. If it exists, aggregate task data and return statistical results.

This kind of requirement is suitable for:

- Decision table testing, because there is a condition branch of "project exists / does not exist" .
- State transition testing, because it can be modeled as "request statistics -> validate project -> return statistical result" .

But it is not naturally suitable for:

- Equivalence partitioning, because there is no explicit input field set or field-level valid/invalid classes.
- Boundary value analysis, because there is no numeric range, length range, or explicit interval suitable for boundary splitting.

From the tool implementation, the EP and BVA generators send the complete `requirement_json` to the LLM and expect it to return:

- `equivalence_classes`
- `boundary_analysis`
- `test_cases`

The corresponding implementations are in `tool/src/test_design/black_box/equivalence_partit` and `tool/src/test_design/black_box/boundary_value.py`.

When the requirement itself has no input fields, ranges, or boundary information, the LLM can usually generate a decision table or a state model more stably, but it is difficult to produce meaningful EP/BVA `test_cases`. The current generators handle this by:

- Allowing empty `test_cases`.
- Recording a warning.
- Showing no cases for that technique on the final page.

Therefore, REQ-007 does not generate EP/BVA cases not because the tool reports an error, but because:

1. The requirement is essentially a read-only statistical requirement with "existence check + aggregation result" .
2. It has no explicit input fields, boundary ranges, or field constraints.
3. Decision table and state transition are more suitable for this type of requirement than EP/BVA.

This also shows that the tool already has a certain level of technique adaptability. When a requirement does not contain field boundary information, the tool does not force low-quality EP/BVA cases, and is more likely to produce results through decision table and state transition.

8 Comparison Between Final Automatic and Manual Scripts

To compare the manually reviewed automatic script with the manual script more directly, the key metrics are summarized below.

Dimension	test_generated.py	test_generated_refined.py	test_human.py
Source	Automatically generated	Automatically generated and then manually reviewed	Written manually from scratch
Number of test functions	44	32	27
Script-level errors	17	0	0
Real defects discovered	0	3 (BUG-7, BUG-8, BUG-9)	1 (BUG-9)
Correction / writing time	About 3-5 minutes for automatic generation	About 20 minutes for review and correction	About 4 hours for manual design and coding
LLM cost	Input 79,260 / output 10,828, about ¥2	Input 79,260 / output 10,828, about ¥2	¥0

Two conclusions can be drawn from this comparison.

First, the main value of the automatic tool is that it provides richer testing perspectives, although the original generated script may still contain some script-level issues.

Second, the manual script is reliable in execution, but its testing perspectives are not as rich as the tool-assisted result after review.

9 Mapping to Known Defects

The known defects related to the task management module are BUG-7, BUG-8, and BUG-9.

Defect	Meaning	Discovery in This Round
BUG-7	Unregistered assignees are not handled correctly during assignment or update	Discovered by <code>test_generated_refined.py</code>
BUG-8	Completed tasks can still be deleted	Discovered by <code>test_generated_refined.py</code>
BUG-9	Past <code>due_date</code> values are not rejected	Discovered by both <code>test_generated_refined.py</code> and <code>test_human.py</code>

Therefore, from the perspective of matching known defect records, the manually reviewed automatic script is clearly better than the manual script.

10 Performance Comparison of Different Models

To understand how the tool behaves under different models, an additional small-scale comparison experiment was conducted on REQ-005 Task Management. The experiment includes one generation round for each of EP, BVA, and DT. It mainly observes generation time and the number of parseable test cases. It does not directly evaluate test case correctness or defect discovery ability.

Model	Configuration	Total Time	Valid Cases	EP	BVA	DT	Judgment
gpt-4o	yunwu.ai / no reasoning parameter	24.414s	22	7.615s / 12 cases	11.060s / 6 cases	5.739s / 4 cases	Most stable in this round
deepseek-v4-flash	thinking disabled	108.415s	3	43.897s / 0 cases	60.405s / 0 cases	4.113s / 3 cases	DT usable, EP/BVA unstable
gemini-3.5-flash	free API	70.940s	14	16.245s / 8 cases	18.400s / 6 cases	36.295s / 0 cases	EP/BVA usable, DT unstable

The results show that gpt-4o has the best speed and parseable output in this round. All three

techniques return structured results that the current tool can recognize. DeepSeek v4 flash is fast in the DT stage, but EP and BVA do not produce effective cases, which suggests that the current workflow is sensitive to output format consistency. Gemini can generate 14 cases in the EP and BVA stages, but the DT stage does not produce valid results, and the free API still has quota-related stability risks. Therefore, its stability is weaker than gpt-4o.

The practical conclusion is direct: if the goal is to complete one full black-box case generation process stably, gpt-4o is more suitable under the current configuration. At the same time, DeepSeek and Gemini also produce usable results in some stages, which shows that the system still keeps flexibility in model selection. In practice, the model can be selected according to speed, cost, and API availability.

11 Generalizability of the System

Based on the task management module experiment, AutoTestDesign is not a script generator hard-coded for the Task Management Platform or for one specific task API. What it really depends on is a more general input and processing workflow: natural language requirements are first organized into structured requirements, then risk analysis, black-box design, white-box code mapping, and script export are performed. In other words, the system focuses on how requirements are transformed into test design, rather than on the API names or business objects of one fixed project.

Therefore, as long as another project can provide relatively clear requirement descriptions and readable code structure, the system can be migrated to that project. For example, in a student management system, it could be used for student registration, course selection, and grade query; in an e-commerce system, it could support product management, order creation, and inventory deduction; in a library system, it could cover borrowing, returning, and overdue handling; and in a permission control system, it could test role assignment and access validation. From this perspective, the generalization target is not the "task management" business itself, but software projects that contain input fields, condition branches, state changes, or API behavior.

During migration, the system can select or help select suitable testing techniques according to requirement characteristics. Requirements involving field validation can use equivalence partitioning and boundary value analysis. Requirements involving condition combinations can use decision tables. Requirements with lifecycle or process changes can use state transition testing. Requirements related to implementation paths can further use white-box path coverage. In this way, even if different projects have different business names, they can still be organized into testable elements such as fields, conditions, behaviors, and code paths.

In addition, the white-box part connects requirements with implementation through requirement-code mapping. If another project has clear APIs, service functions, and data models, the system

can continue to generate path coverage results and help testers supplement paths that may be missed by black-box testing. Overall, the system has cross-project migration value and is suitable as a general tool for generating test design drafts and supporting analysis.

12 Conclusion

This detailed test design and execution report leads to the following conclusions.

1. AutoTestDesign already shows clear advantages in the test design stage. It can quickly produce black-box analysis tables, white-box coverage ideas, and large-scale candidate test scripts. This is difficult to achieve with a purely manual approach in a short time.
2. Automatically generated scripts cannot completely skip the manual step. The original exported script has many problems, but after one limited manual review, script quality improves significantly and defect discovery ability is also strengthened.
3. Based on this round of actual results, the more reasonable workflow is not "automation replacing humans", but "automation generating the first draft and humans performing key review". This approach achieves a better balance between time cost and defect discovery ability.
4. Model selection directly affects the stability of the tool. In the small-scale REQ-005 experiment, gpt-4o is more suitable for the current workflow in terms of speed and valid case output. If other models are integrated, the first problem to solve is the stability of structured output and exception retry.
5. The system has cross-project generalizability and is not limited to the Task Management Platform used in this experiment. As long as another project provides clear requirements and a code structure that can be mapped, the tool can be migrated to generate risk analysis, test design, and initial scripts. For abstract requirements or complex code mappings, manual review and testing-technique selection are still necessary.

The related materials for this round include:

- docs/01_risk_analysis_report.md
- docs/02_test_plan.md
- test_scripts/test_generated.py
- test_scripts/test_generated_refined.py
- test_scripts/test_human.py
- target_app/flask_app/planted_bugs.md
- run_results/REQ005_006_007/